

SQL Business Domain Practice Set – Answers

Snowflake-Compatible SQL | 10 Scenario-Based Questions

Q1 | Retail – Customer-wise Net Spending on Completed Orders

Problem Statement

Management wants to identify customers who generated the highest net sales value from delivered orders. The calculation must account for item quantity, unit price, and discount percentage. Only orders with status `Delivered` are considered, and net line amount = `quantity * unit_price * (1 - discount_pct / 100)` . Customers must be displayed with the highest spending first.

Data

`retail_customers`

customer_id	customer_name	city
1	Aarav Mehta	Pune
2	Diya Nair	Chennai
3	Kabir Shah	Mumbai
4	Meera Rao	Bengaluru
5	Rohan Das	Kolkata

`retail_orders`

order_id	customer_id	order_date	order_status
1001	1	2026-01-05	Delivered
1002	2	2026-01-08	Delivered
1003	1	2026-02-02	Delivered
1004	3	2026-02-10	Cancelled
1005	4	2026-02-15	Delivered
1006	5	2026-03-01	Pending

order_id	customer_id	order_date	order_status
1007	3	2026-03-05	Delivered

retail_order_items

item_id	order_id	product_name	quantity	unit_price	discount_pct
1	1001	Laptop Stand	1	2500	10
2	1001	Wireless Mouse	2	900	0
3	1002	Office Chair	1	7200	5
4	1002	Desk Lamp	2	1200	0
5	1003	Keyboard	1	1800	0
6	1003	USB Hub	2	1400	10
7	1004	Monitor	1	15000	0
8	1005	Bookshelf	1	8500	10
9	1005	Desk Lamp	1	1200	0
10	1006	Office Chair	1	7200	0
11	1007	Monitor	1	15000	8
12	1007	Wireless Mouse	1	900	0

Query

```
SELECT
    a.customer_name,
    a.city,
    SUM(c.quantity * c.unit_price * (1 - c.discount_pct / 100)) AS total_sp
FROM retail_customers a
JOIN retail_orders b
    ON a.customer_id = b.customer_id
JOIN retail_order_items c
    ON b.order_id = c.order_id
WHERE b.order_status = 'Delivered'
GROUP BY
    a.customer_name,
    a.city
ORDER BY
    total_spent DESC;
```

Output

customer_name	city	total_spent
Kabir Shah	Mumbai	14700.00
Diya Nair	Chennai	9240.00
Meera Rao	Bengaluru	8850.00
Aarav Mehta	Pune	8370.00

Q2 | Banking – Branch-wise Successful Loan Collection Analysis

Problem Statement

The collections team wants to compare branches by the amount of successful loan repayments collected, and also see how many distinct borrowers contributed to those successful collections. Only repayment rows with `payment_status = SUCCESS` are considered, each borrower is counted once per branch, and branches are displayed with the highest collected amount first.

Data

bank_branches

branch_id	branch_name	city
1	Central Branch	Chennai
2	Tech Park Branch	Bengaluru
3	Metro Branch	Hyderabad

bank_customers

customer_id	customer_name	branch_id
101	Vikram Iyer	1
102	Nisha Kapoor	1
103	Arjun Menon	2
104	Pooja Singh	2
105	Sameer Khan	3
106	Latha Reddy	3

bank_loans

loan_id	customer_id	loan_type	principal_amount
5001	101	Home	3500000
5002	102	Personal	500000
5003	103	Vehicle	900000
5004	104	Home	4200000
5005	105	Business	2500000
5006	106	Personal	600000

bank_repayments

repayment_id	loan_id	payment_date	amount_paid	payment_status
1	5001	2026-01-10	45000	SUCCESS
2	5001	2026-02-10	45000	SUCCESS
3	5002	2026-01-18	18000	SUCCESS
4	5002	2026-02-18	18000	FAILED
5	5003	2026-01-12	26000	SUCCESS
6	5003	2026-02-12	26000	SUCCESS
7	5004	2026-01-20	52000	SUCCESS
8	5004	2026-02-20	52000	SUCCESS
9	5005	2026-01-25	70000	SUCCESS
10	5005	2026-02-25	70000	FAILED
11	5006	2026-01-28	21000	SUCCESS
12	5006	2026-02-28	21000	SUCCESS

Query

```
SELECT
    a.branch_name,
    a.city,
    COUNT(DISTINCT b.customer_id) AS borrower_count,
    SUM(d.amount_paid) AS total_collected
FROM bank_branches a
JOIN bank_customers b
    ON a.branch_id = b.branch_id
JOIN bank_loans c
    ON b.customer_id = c.customer_id
```

```
JOIN bank_repayments d
  ON c.loan_id = d.loan_id
WHERE d.payment_status = 'SUCCESS'
GROUP BY
  a.branch_name,
  a.city
ORDER BY
  total_collected DESC;
```

Output

branch_name	city	borrower_count	total_collected
Tech Park Branch	Bengaluru	2	156000.00
Metro Branch	Hyderabad	2	112000.00
Central Branch	Chennai	2	108000.00

Q3 | Healthcare – Doctor-wise Completed Consultation Revenue

Problem Statement

The hospital operations team wants to compare doctors by completed consultation revenue and the number of unique patients served. Cancelled appointments and no-shows must not contribute to the result. Only appointments with status **Completed** are used, each patient is counted once per doctor, and doctors are displayed with the highest revenue first.

Data

doctors

doctor_id	doctor_name	specialty
1	Dr. Ananya Rao	Cardiology
2	Dr. Karthik Bose	Orthopedics
3	Dr. Sana Ali	Dermatology
4	Dr. Vivek Nair	Pediatrics

patients

patient_id	patient_name	city
------------	--------------	------

patient_id	patient_name	city
201	Ishaan	Chennai
202	Tara	Chennai
203	Nakul	Bengaluru
204	Aditi	Hyderabad
205	Reyansh	Chennai
206	Maya	Pune

appointments

appointment_id	doctor_id	patient_id	appointment_date	consultation_fee	appointment_status
9001	1	201	2026-04-01	1200	Completed
9002	1	202	2026-04-02	1200	Completed
9003	1	201	2026-04-15	1200	Completed
9004	2	203	2026-04-03	900	Completed
9005	2	204	2026-04-10	900	Cancelled
9006	2	205	2026-04-12	900	Completed
9007	3	204	2026-04-05	800	Completed
9008	3	206	2026-04-18	800	Completed
9009	4	205	2026-04-06	700	Completed
9010	4	201	2026-04-20	700	Completed
9011	4	202	2026-04-25	700	No Show

Query

```
SELECT
    a.doctor_name,
    a.specialty,
    COUNT(DISTINCT b.patient_id) AS unique_patients,
    SUM(b.consultation_fee) AS consultation_revenue
FROM doctors a
JOIN appointments b
    ON a.doctor_id = b.doctor_id
WHERE b.appointment_status = 'Completed'
GROUP BY
    a.doctor_name,
    a.specialty
```

```
ORDER BY
consultation_revenue DESC;
```

Output

doctor_name	specialty	unique_patients	consultation_revenue
Dr. Ananya Rao	Cardiology	2	3600.00
Dr. Karthik Bose	Orthopedics	2	1800.00
Dr. Sana Ali	Dermatology	2	1600.00
Dr. Vivek Nair	Pediatrics	2	1400.00

Q4 | Telecom – Plan-wise Monthly Data Usage Revenue

Problem Statement

The telecom finance team wants month-wise data usage revenue for every plan, calculated as billed data volume multiplied by the plan rate per GB. Only usage rows with status **BILLED** are considered, and results are displayed plan-wise and then month-wise.

Data

```
telecom_plans
```

plan_id	plan_name	data_rate_per_gb
1	Smart 399	12.00
2	Max 599	9.00
3	Ultra 799	7.50

```
telecom_subscribers
```

subscriber_id	subscriber_name	plan_id
301	Aditya	1
302	Bhavna	1
303	Charan	2
304	Deepa	2
305	Farhan	3
306	Gauri	3

usage_id	subscriber_id	usage_date	data_gb	usage_status
1	301	2026-01-05	18	BILLED
2	302	2026-01-08	12	BILLED
3	303	2026-01-10	25	BILLED
4	304	2026-01-15	20	BILLED
5	305	2026-01-18	40	BILLED
6	306	2026-01-20	32	BILLED
7	301	2026-02-05	22	BILLED
8	302	2026-02-08	10	BILLED
9	303	2026-02-10	28	BILLED
10	304	2026-02-15	24	REVERSED
11	305	2026-02-18	44	BILLED
12	306	2026-02-20	36	BILLED

Query

```
SELECT
    a.plan_name,
    DATE_TRUNC('MONTH', c.usage_date) AS month_start,
    SUM(c.data_gb * a.data_rate_per_gb) AS usage_revenue
FROM telecom_plans a
JOIN telecom_subscribers b
    ON a.plan_id = b.plan_id
JOIN telecom_usage c
    ON b.subscriber_id = c.subscriber_id
WHERE c.usage_status = 'BILLED'
GROUP BY
    a.plan_name,
    DATE_TRUNC('MONTH', c.usage_date)
ORDER BY
    a.plan_name,
    month_start;
```

Output

plan_name	month_start	usage_revenue

plan_name	month_start	usage_revenue
Max 599	2026-01-01	405.00
Max 599	2026-02-01	252.00
Smart 399	2026-01-01	360.00
Smart 399	2026-02-01	384.00
Ultra 799	2026-01-01	540.00
Ultra 799	2026-02-01	600.00

Q5 | E-commerce – Products Selling Above the Overall Average Quantity

Problem Statement

Merchandising wants to identify products whose completed-order sales quantity is greater than the average quantity sold across all products. Only **Completed** orders are used; the total quantity sold is determined per product first, and only products exceeding the average of those totals are returned, displayed highest quantity first.

Data

ecom_products

product_id	product_name	category
401	Noise Cancelling Headphones	Electronics
402	Fitness Band	Electronics
403	Running Shoes	Sports
404	Yoga Mat	Sports
405	Coffee Maker	Home
406	Air Fryer	Home

ecom_orders

order_id	order_date	order_status
7001	2026-03-01	Completed
7002	2026-03-02	Completed
7003	2026-03-03	Cancelled

order_id	order_date	order_status
7004	2026-03-04	Completed
7005	2026-03-05	Completed

ecom_order_items

item_id	order_id	product_id	quantity
1	7001	401	3
2	7001	403	2
3	7002	402	5
4	7002	404	4
5	7003	406	10
6	7004	401	2
7	7004	405	6
8	7005	403	5
9	7005	406	3
10	7005	404	1

Query

```
WITH product_totals AS (  
  SELECT  
    a.product_name,  
    SUM(b.quantity) AS total_quantity_sold  
  FROM ecom_products a  
  JOIN ecom_order_items b  
    ON a.product_id = b.product_id  
  JOIN ecom_orders c  
    ON b.order_id = c.order_id  
  WHERE c.order_status = 'Completed'  
  GROUP BY  
    a.product_name  
)  
SELECT  
  product_name,  
  total_quantity_sold  
FROM product_totals  
WHERE total_quantity_sold > (  
  SELECT AVG(total_quantity_sold)
```

```
FROM product_totals
)
ORDER BY
    total_quantity_sold DESC;
```

Output

product_name	total_quantity_sold
Running Shoes	7
Coffee Maker	6

Q6 | Insurance – Policyholders with Above-Average Approved Claims

Problem Statement

Risk management wants to identify policyholders whose total approved claim amount is greater than the average approved claim total across policyholders. Only claims with status **Approved** are used, approved claims are aggregated across all policies owned by the same customer, and only customers above the average customer approved-claim total are returned, displayed highest total first.

Data

ins_customers

customer_id	customer_name	city
501	Harini	Chennai
502	Jatin	Delhi
503	Keerthi	Bengaluru
504	Lokesh	Pune
505	Manasa	Hyderabad

ins_policies

policy_id	customer_id	policy_type
8001	501	Health
8002	501	Motor
8003	502	Health

policy_id	customer_id	policy_type
8004	503	Motor
8005	504	Health
8006	505	Motor

ins_claims

claim_id	policy_id	claim_date	claim_amount	claim_status
1	8001	2026-01-11	45000	Approved
2	8002	2026-02-04	18000	Approved
3	8003	2026-01-18	25000	Approved
4	8003	2026-03-01	12000	Rejected
5	8004	2026-02-15	72000	Approved
6	8005	2026-01-30	30000	Approved
7	8005	2026-03-10	15000	Approved
8	8006	2026-02-22	20000	Approved

Query

```
WITH customer_totals AS (  
  SELECT  
    a.customer_name,  
    SUM(c.claim_amount) AS total_approved_claim  
  FROM ins_customers a  
  JOIN ins_policies b  
    ON a.customer_id = b.customer_id  
  JOIN ins_claims c  
    ON b.policy_id = c.policy_id  
  WHERE c.claim_status = 'Approved'  
  GROUP BY  
    a.customer_name  
)  
SELECT  
  customer_name,  
  total_approved_claim  
FROM customer_totals  
WHERE total_approved_claim > (  
  SELECT AVG(total_approved_claim)  
  FROM customer_totals  
)
```

```
ORDER BY
    total_approved_claim DESC;
```

Output

customer_name	total_approved_claim
Keerthi	72000.00
Harini	63000.00

Q7 | Manufacturing – Machines Costing Above Their Type Average

Problem Statement

Procurement wants to identify machines whose purchase cost is higher than the average purchase cost of machines belonging to the same machine type, using a strict greater-than comparison, displayed by machine type and then highest cost.

Data

factory_machines

machine_id	machine_name	machine_type	purchase_cost	plant
601	CNC Alpha	CNC	4200000	Chennai
602	CNC Beta	CNC	3600000	Pune
603	CNC Gamma	CNC	3000000	Coimbatore
604	Press One	Hydraulic Press	2800000	Chennai
605	Press Two	Hydraulic Press	2200000	Pune
606	Press Three	Hydraulic Press	2000000	Coimbatore
607	Robot X	Welding Robot	5100000	Chennai
608	Robot Y	Welding Robot	4500000	Pune
609	Robot Z	Welding Robot	3900000	Coimbatore

Query

```
SELECT
    a.machine_name,
    a.machine_type,
```

```

a.purchase_cost
FROM factory_machines a
WHERE a.purchase_cost > (
    SELECT AVG(b.purchase_cost)
    FROM factory_machines b
    WHERE b.machine_type = a.machine_type
)
ORDER BY
    a.machine_type,
    a.purchase_cost DESC;
```

Output

machine_name	machine_type	purchase_cost
CNC Alpha	CNC	4200000.00
Press One	Hydraulic Press	2800000.00
Robot X	Welding Robot	5100000.00

Q8 | Airline – Monthly Confirmed Booking Revenue Trend

Problem Statement

Commercial management wants a month-wise report showing confirmed booking revenue, the previous month's revenue, and the difference between the current and previous month. Only **Confirmed** bookings are used, the first month shows **NULL** for previous month and difference, and months are displayed chronologically.

Data

air_flights

flight_id	route
901	Chennai-Delhi
902	Bengaluru-Mumbai
903	Hyderabad-Kolkata

air_bookings

booking_id	flight_id	booking_date	fare_amount	booking_status
1	901	2026-01-05	8200	Confirmed

booking_id	flight_id	booking_date	fare_amount	booking_status
2	902	2026-01-12	6400	Confirmed
3	903	2026-01-20	7200	Cancelled
4	901	2026-02-03	8500	Confirmed
5	902	2026-02-10	6700	Confirmed
6	903	2026-02-18	7600	Confirmed
7	901	2026-03-04	9000	Confirmed
8	902	2026-03-11	6900	Cancelled
9	903	2026-03-21	7800	Confirmed
10	901	2026-04-02	8800	Confirmed
11	902	2026-04-14	7100	Confirmed
12	903	2026-04-22	8000	Confirmed

Query

```
WITH monthly_revenue AS (  
    SELECT  
        DATE_TRUNC('MONTH', booking_date) AS month_start,  
        SUM(fare_amount) AS total_revenue  
    FROM air_bookings  
    WHERE booking_status = 'Confirmed'  
    GROUP BY  
        DATE_TRUNC('MONTH', booking_date)  
)  
SELECT  
    month_start,  
    total_revenue,  
    LAG(total_revenue) OVER (ORDER BY month_start) AS previous_month_revenue,  
    total_revenue - LAG(total_revenue) OVER (ORDER BY month_start) AS revenue_difference  
FROM monthly_revenue  
ORDER BY  
    month_start;
```

Output

month_start	total_revenue	previous_month_revenue	revenue_difference
2026-01-01	14600.00	NULL	NULL
2026-02-01	22800.00	14600.00	8200.00

month_start	total_revenue	previous_month_revenue	revenue_difference
2026-03-01	16800.00	22800.00	-6000.00
2026-04-01	23900.00	16800.00	7100.00

Q9 | Logistics – Client Shipment Cost versus Previous Delivery

Problem Statement

Operations wants to compare each delivered shipment cost with the immediately previous delivered shipment cost for the same client. Only `Delivered` shipments are considered, the previous shipment is determined independently per client, and the first delivered shipment per client shows `NULL` for previous cost and difference.

Data

`log_clients`

client_id	client_name
701	Apex Retail
702	BlueMart
703	Crest Pharma

`log_shipments`

shipment_id	client_id	shipped_date	delivery_cost	shipment_status
10001	701	2026-05-01	12500	Delivered
10002	701	2026-05-15	13800	Delivered
10003	701	2026-06-02	13100	Delivered
10004	702	2026-05-03	9800	Delivered
10005	702	2026-05-19	10200	Cancelled
10006	702	2026-06-05	11000	Delivered
10007	703	2026-05-07	15600	Delivered
10008	703	2026-05-22	14900	Delivered
10009	703	2026-06-09	16200	Delivered

Query


```

SELECT
    a.client_name,
    b.shipment_id,
    b.shipped_date,
    b.delivery_cost,
    LAG(b.delivery_cost) OVER (PARTITION BY a.client_id ORDER BY b.shipped_date) AS previous_cost,
    b.delivery_cost - LAG(b.delivery_cost) OVER (PARTITION BY a.client_id ORDER BY b.shipped_date) AS cost_difference
FROM log_clients a
JOIN log_shipments b
    ON a.client_id = b.client_id
WHERE b.shipment_status = 'Delivered'
ORDER BY
    a.client_name,
    b.shipped_date;

```

Output

client_name	shipment_id	shipped_date	delivery_cost	previous_cost	cost_difference
Apex Retail	10001	2026-05-01	12500.00	NULL	NULL
Apex Retail	10002	2026-05-15	13800.00	12500.00	1300.00
Apex Retail	10003	2026-06-02	13100.00	13800.00	-700.00
BlueMart	10004	2026-05-03	9800.00	NULL	NULL
BlueMart	10006	2026-06-05	11000.00	9800.00	1200.00
Crest Pharma	10007	2026-05-07	15600.00	NULL	NULL
Crest Pharma	10008	2026-05-22	14900.00	15600.00	-700.00
Crest Pharma	10009	2026-06-09	16200.00	14900.00	1300.00

Q10 | Human Resources – Employee Salary with Department Payroll Contribution

Problem Statement

HR leadership wants to see each active employee's salary alongside the total active payroll of the employee's department, the employee's percentage contribution to that payroll, and salary rank within the department. Only **Active** employees are used, payroll contribution % = $\text{salary} / \text{department payroll} * 100$, higher salaries rank first within each department, and results are displayed department-wise and then salary-rank-wise.

Data

hr_departments

dept_id	dept_name
1	Data Engineering
2	Finance
3	Sales

hr_employees

emp_id	emp_name	dept_id	salary	employment_status
801	Akhil	1	95000	Active
802	Bhavya	1	85000	Active
803	Chetan	1	70000	Active
804	Divya	2	90000	Active
805	Eshan	2	75000	Active
806	Fatima	2	65000	Exited
807	Girish	3	80000	Active
808	Hema	3	70000	Active
809	Imran	3	60000	Active

Query

```
SELECT
    a.dept_name,
    b.emp_name,
    b.salary,
    SUM(b.salary) OVER (PARTITION BY a.dept_id) AS dept_payroll,
    ROUND(b.salary / SUM(b.salary) OVER (PARTITION BY a.dept_id) * 100, 2) .
    RANK() OVER (PARTITION BY a.dept_id ORDER BY b.salary DESC) AS salary_r
FROM hr_departments a
JOIN hr_employees b
    ON a.dept_id = b.dept_id
WHERE b.employment_status = 'Active'
ORDER BY
    a.dept_name,
    salary_rank;
```

Output

dept_name	emp_name	salary	dept_payroll	contribution_pct	salary_rank
Data Engineering	Akhil	95000.00	250000.00	38.00	1
Data Engineering	Bhavya	85000.00	250000.00	34.00	2
Data Engineering	Chetan	70000.00	250000.00	28.00	3
Finance	Divya	90000.00	165000.00	54.55	1
Finance	Eshan	75000.00	165000.00	45.45	2
Sales	Girish	80000.00	210000.00	38.10	1
Sales	Hema	70000.00	210000.00	33.33	2
Sales	Imran	60000.00	210000.00	28.57	3